CO215, Assignment 8

Roll number: CSB23033

Name: Niruj nayan

saikia

```
.MODEL SMALL
.STACK 100H

.DATA
array     DWMAX_SIZEDUP(?)
prompt_msg DB 'Enter single digit non-zero integers to be sorted.', CR, LF
          DB 'Enter zero to terminate the input (at most 10).', CR, LF, '$'
output_msg DB 'Array of numbers in sorted order:', CR, LF, '$'
nwln      DBCR,LF,'$'
count     DW?
temp      DW?

.CODE

main PROC
   MOV AX, @DATA
   MOV DS, AX

   MOV AH, 09H
   LEA DX, prompt_msg
   INT 21H
```

```
    LEA BX, array
    XOR CX, CX

input_loop:
    MOV AH, 01H
    INT 21H
    SUB AL, '0'
    MOV AH, 0

    CMP AX, 0
    JE done_input

    MOV [BX], AX
    ADD BX, 2
    INC CX
    CMP CX, MAX_SIZE
    JL input_loop

done_input:
    MOV count, CX

    CALL selection_sort

    MOV AH, 09H
    LEA DX, output_msg
    INT 21H

    CMP count, 0
    JE exit_program

    LEA BX, array
```

```
    MOV CX, count

print_loop:
    MOV DL, BYTE PTR [BX]
    ADD DL, '0'
    MOV AH, 02H
    INT 21H

    MOV AH, 09H
    LEA DX, nwln
    INT 21H

    ADD BX, 2
    LOOP print_loop

exit_program:
    MOV AH, 4CH
    MOV AL, 0
    INT 21H
main ENDP

selection_sort PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    PUSH SI
    PUSH DI

    MOV CX, count
    CMP CX, 1
```

```
    JLE sort_done

    XOR SI, SI

outer_loop:
    MOV DI, SI
    MOV AX, SI
    INC AX

inner_loop:
    CMP AX, count
    JGE swap_elements

    MOV BX, OFFSET array
    MOV DX, DI
    SHL DX, 1
    ADD BX, DX
    MOV DX, [BX]

    MOV BX, OFFSET array
    PUSH DX
    MOV DX, AX
    SHL DX, 1
    ADD BX, DX
    MOV DX, [BX]
    MOV BX, DX
    POP DX

    CMP BX, DX
    JGE next_element
```

```
    MOV DI, AX

next_element:
    INC AX
    JMP inner_loop

swap_elements:
    CMP DI, SI
    JE next_iteration

    MOV BX, OFFSET array
    MOV DX, SI
    SHL DX, 1
    ADD BX, DX
    MOV DX, [BX]
    MOV temp, DX

    MOV BX, OFFSET array
    MOV DX, DI
    SHL DX, 1
    ADD BX, DX
    MOV DX, [BX]

    MOV BX, OFFSET array
    PUSH DX
    MOV DX, SI
    SHL DX, 1
    ADD BX, DX
    POP DX
    MOV [BX], DX
```

```
    MOV BX, OFFSET array
    MOV DX, DI
    SHL DX, 1
    ADD BX, DX
    MOV DX, temp
    MOV [BX], DX

next_iteration:
    INC SI
    MOV AX, count
    CMP SI, AX
    JL outer_loop

sort_done:
    POP DI
    POP SI
    POP DX
    POP CX
    POP BX
    POP AX
    RET
selection_sort ENDP

END main
```

Observations:

Selection_Sort		
Instruction No	**Starting memory location**	**Machine Code in Hexadecimal**
1	0000	**BA8C08**
2	0003	**8EDA**

3	0005	**8CD5**
4	0007	**2BDA**
5	0009	**D1E3**
6	000B	**D1E3**
7	000D	**D1E3**
8	000F	**D1E3**
9	0011	**FA**
10	0012	**8ED2**
11	0014	**BC00**
12	0016	**FB**
13	0017	**B409**
14	0019	**BA1400**
15	001C	**CD21**
16	001E	**BB0000**
17	0021	**B90A00**
18	0024	**2BD2**
19	0026	**B401**
20	0028	**CD21**
21	002A	**2C30**
22	002C	**B400**
23	002E	**3D0000**
24	0031	**740C**
25	0033	**8907**
26	0035	**83C302**
27	0038	**42**
28	0039	**E2EB**
29	003B	**89160000**
30	003F	**E84C00**
31	0042	**B409**
32	0044	**BA3E00**

33	0047	**CD21**
34	0049	**8B0E0000**
35	004D	**0BCE**
36	004F	**7414**
37	0051	**BB0000**
38	0054	**B402**
39	0056	**8B17**
40	0058	**80C230**
41	005B	**CD21**
42	005D	**B409**
43	005F	**BA5A00**
44	0062	**CD21**
45	0064	**83C302**
46	0067	**E2EB**
47	0069	**B44C**
48	006B	**B000**
49	006D	**CD21**
50	006F	**50**
51	0070	**53**
52	0071	**51**
53	0072	**52**
54	0073	**56**
55	0074	**57**
56	0075	**8B0E0000**
57	0079	**83F901**
58	007C	**7E47**
59	007E	**BE0000**
60	0081	**8BFE**
61	0083	**8BC6**
62	0085	**40**

63	0086	**3BC1**
64	0088	**7D21**
65	008A	**BB0000**
66	008D	**8BD7**
67	008F	**03D2**
68	0091	**03DA**
69	0093	**8B17**
70	0095	**BB0000**
71	0098	**52 8BD0**
72	0099	**03D2**
73	009B	**03DA**
74	009D	**8B17**
75	009F	**8BDA 5A**
76	00A1	**3BDA**
77	00A3	**7D02**
78	00A4	**8BF8 40**
79	00A6	**EBDE**
80	00A8	**3BFE**
81	00AA	**7418**
82	00AB	**BB0000**
83	00AD	**8BD6**
84	00AF	**03D2**
85	00B1	**03DA**
86	00B4	**8B17**
87	00B6	**89160000**
88	00B8	**BB0000**
89	00BA	**8BD7**
90	00BC	
91	00C0	
92	00C3	

93	00C5	**03D2**
94	00C7	**03DA**
95	00C9	**8B17**
96	00CB	**BB0000**
97	00CE	**52**
98	00CF	**8BD6**
99	00D1	**03D2**
100	00D3	**03DA**
101	00D5	**5A**
102	00D6	**8917**
103	00D8	**BB0000**
104	00DB	**8BD7**
105	00DD	**03D2**
106	00DF	**03DA**
107	00E1	**8B160000**
108	00E5	**8917**
109	00E7	**46**
110	00E8	**3BF1**
111	00EA	**7C95**
112	00EC	**5F**
113	00ED	**5E**
114	00EE	**5A**
115	00EF	**59**
116	00F0	**5B**
117	00F1	**58**
118	00F2	**C3**

Bubble_Sort		
Instruction No.	**Starting memory location**	**Machine Code in Hexadecimal**
1	0000	BA8C08

2	0003	8EDA
3	0005	8CD5
4	0007	2BDA
5	0009	D1E3
6	000B	D1E3
7	000D	D1E3
8	000F	D1E3
9	0011	FA
10	0012	8ED2
11	0014	BC00
12	0016	FB
13	0017	B409
14	0019	BA1400
15	001C	CD21
16	001E	BB0000
17	0021	B90A00
18	0024	2BD2
19	0026	B401
20	0028	CD21
21	002A	2C30
22	002C	B400
23	002E	3D0000
24	0031	740A
25	0033	8907
26	0035	83C302
27	0038	42
28	0039	E2EB
29	003B	52
30	003C	8BCA
31	003E	68----
32	0041	E8----

33	0044	B409
34	0046	BA----
35	0049	CD21
36	004B	BB0000
37	004E	B402
38	0050	8B17
39	0052	80C230
40	0055	CD21
41	0057	B409
42	0059	BA----
43	005C	CD21
44	005E	83C302
45	0061	E2EB
46	0063	B44C
47	0065	B000
48	0067	CD21
49	0069	50
50	006A	53
51	006B	51
52	006C	52
53	006D	54
54	006E	56
55	006F	57
56	0070	55
57	0071	8BEC
58	0073	8B4E14
59	0076	8B5E12
60	0079	49
61	007A	7428
62	007C	8BF9
63	007E	BA0000

64	0081	8BF3
65	0083	8B04
66	0085	3B4402
67	0088	7F0A
68	008A	83C602
69	008D	4F
70	008E	75F3
71	0090	83FA00
72	0093	740F
73	0095	EBE2
74	0097	874402
75	009A	8904
76	009C	BA0100
77	009F	EBEB
78	00A1	5D 5F
79	00A2	5E 5C
80	00A3	5A 59
81	00A4	5B 58
82	00A5	C20400
83	00A6	
84	00A7	
85	00A8	
86	00A9	

Comparison:

Program	**Nmos troufctions**	**Peronggrtahm (Bytes)**	**ONupmerbaetiroonfsMemory**	**Remarks**
		116		
Bubble Sort	54	121	72	Register Indirect Addressing Mode
Selection Sort	53		79	Register Indexed Addressing Mode

Learning Outcome:

1. **ParameterPassing**:Welearnhowdataorvaluesarepassedfromthemainprogramtoa procedure, allowing the procedure to work with different inputs without rewriting the code.
2. **UseofAlternativeAddressingModes**:Weunderstandhowdifferentaddressingmethods (like immediate, direct, register, and indirect) help access and manipulate data stored in various memory locations efficiently.
3. **ChangeofCSandIPvaluesduringprocedurecallandreturn**:WeobservehowtheCode Segment (CS) and Instruction Pointer (IP) registers are updated when a procedure is called, and how they are restored to their original values when the procedure finishes and returns control to the main program.